Lecture 2: Linear Neural Networks for Regression

Course Teacher: Md. Shariful Islam Bhuyan

Chapter Overview

Topics Covered:

1. Linear Regression Basics
2. Loss Functions & Optimization
3. Analytical Solution (Detailed)
4. Gradient Descent Methods
5. Implementation from Scratch
6. Generalization & Regularization

Why Start Here?

- Focus on **fundamentals** before complex architectures
- Linear models $\subset$ neural networks
- Introduces key concepts:
  - Parameterization
  - Loss functions
  - Optimization
  - Training loops
- Baseline for justifying fancier models

Part I

Linear Regression Basics

The Simplest Neural Network

What is Regression?

Definition

Regression predicts a continuous numerical value based on input features.

Examples:

- Predicting **house prices** based on area, age, location
- Forecasting **stock prices** from financial indicators
- Estimating **hospital stay duration** for patients
- Predicting **demand** for retail products

Terminology:

- **Training set** The dataset used to learn the model
- **Example/Sample** One row of data (features + label)
- **Features** Input variables (also: covariates, inputs)
- **Label/Target** The value we want to predict

The Linear Model

Core Assumption

The target y is approximately a **weighted sum** of features plus noise.

House Price Example:

$$\text{price} = w_{\text{area}} \cdot \text{area} + w_{\text{age}} \cdot \text{age} + b$$

General Form (single example with d features):

$$\hat{y} = w_1 x_1 + w_2 x_2 + \cdots + w_d x_d + b$$

Compact Vector Notation:

$$\boxed{\hat{y} = \mathbf{w}^\top \mathbf{x} + b}$$

where:

- $\mathbf{x} \in \mathbb{R}^d$ — feature vector
- $\mathbf{w} \in \mathbb{R}^d$ — **weights** (influence of each feature)
- $b \in \mathbb{R}$ — **bias** (intercept/offset)

Matrix Formulation for Multiple Examples

For n examples, stack all features into a **design matrix**:

$$\mathbf{X} = \begin{bmatrix} x_1^{(1)} & x_2^{(1)} & \cdots & x_d^{(1)} \\ x_1^{(2)} & x_2^{(2)} & \cdots & x_d^{(2)} \\ \vdots & \vdots & \ddots & \vdots \\ x_1^{(n)} & x_2^{(n)} & \cdots & x_d^{(n)} \end{bmatrix} \in \mathbb{R}^{n \times d}$$

All predictions at once:

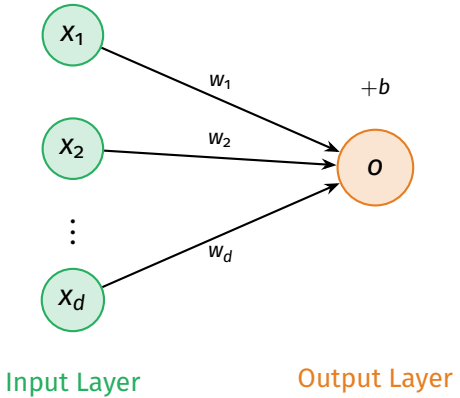
$$\boxed{\hat{\mathbf{y}} = \mathbf{X}\mathbf{w} + b}$$

where $\hat{\mathbf{y}} \in \mathbb{R}^n$ is the vector of all predictions.

Goal

Find $\mathbf{w}^*$ and b^* such that predictions $\hat{\mathbf{y}}$ are as close as possible to true labels $\mathbf{y}$.

Linear Regression as a Neural Network



Key Observations:

- **Single-layer** network
- Every input connects to output
- Called **fully connected** or **dense** layer
- Output: $o = \sum_{i=1}^{d} w_i x_i + b$

Insight: Linear regression is the simplest neural network—inputs directly wired to outputs!

Part II

Loss Function & Optimization

Measuring and Minimizing Error

The Squared Error Loss Function

Loss for a Single Example

$$\ell^{(i)}(\mathbf{w}, b) = \frac{1}{2}\left(\hat{y}^{(i)} - y^{(i)}\right)^2$$

Why squared error?

- Always non-negative
- Differentiable everywhere
- Penalizes large errors heavily (quadratic)
- The $\frac{1}{2}$ cancels nicely when we differentiate

Average Loss over All Examples:

$$\boxed{L(\mathbf{w}, b) = \frac{1}{n}\sum_{i=1}^{n} \ell^{(i)}(\mathbf{w}, b) = \frac{1}{n}\sum_{i=1}^{n} \frac{1}{2}\left(\mathbf{w}^\top \mathbf{x}^{(i)} + b - y^{(i)}\right)^2}$$

Goal: Find optimal parameters

Probabilistic Interpretation: Maximum Likelihood

Assumption: Gaussian Noise

Observations arise from the true linear relationship plus Gaussian noise:

$$y = \mathbf{w}^\top \mathbf{x} + b + \epsilon \quad \text{where } \epsilon \sim \mathcal{N}(0, \sigma^2)$$

Likelihood of observing y given x:

$$P(y \mid \mathbf{x}) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(y - \mathbf{w}^\top \mathbf{x} - b)^2}{2\sigma^2}\right)$$

Log-likelihood for all data (assuming independence):

$$\log P(\mathbf{y} \mid \mathbf{X}) = \underbrace{-\frac{n}{2}\log(2\pi\sigma^2)}_{\text{Constant}} - \frac{1}{2\sigma^2}\sum_{i=1}^{n}(y^{(i)} - \mathbf{w}^\top \mathbf{x}^{(i)} - b)^2$$

Key Result

Maximizing log-likelihood $\Leftrightarrow$ Minimizing squared error loss

The squared loss emerges naturally from the Gaussian noise assumption!

Part III

Analytical Solution

The Normal Equations — Complete Derivation

Setting Up the Analytical Solution

Step 1: Absorb bias into weights

Append a column of 1s to the design matrix:

$$\tilde{\mathbf{X}} = \begin{bmatrix} \mathbf{X} & \mathbf{1} \end{bmatrix} = \begin{bmatrix} x_1^{(1)} & \cdots & x_d^{(1)} & 1 \\ x_1^{(2)} & \cdots & x_d^{(2)} & 1 \\ \vdots & \ddots & \vdots & \vdots \\ x_1^{(n)} & \cdots & x_d^{(n)} & 1 \end{bmatrix} \in \mathbb{R}^{n \times (d+1)}$$

New parameter vector: $\tilde{\mathbf{w}} = \begin{bmatrix} \mathbf{w} \\ b \end{bmatrix} \in \mathbb{R}^{d+1}$

Step 2: Express loss in matrix form

$$L(\tilde{\mathbf{w}}) = \frac{1}{2n} \|\mathbf{y} - \tilde{\mathbf{X}}\tilde{\mathbf{w}}\|^2 = \frac{1}{2n} (\mathbf{y} - \tilde{\mathbf{X}}\tilde{\mathbf{w}})^\top (\mathbf{y} - \tilde{\mathbf{X}}\tilde{\mathbf{w}})$$

For simplicity, we'll drop the tilde and work with:

$$L(\mathbf{w}) = \frac{1}{2n} \|\mathbf{y} - \mathbf{X}\mathbf{w}\|^2$$

Expanding the Squared Loss

Expand the squared norm:

$$
\begin{aligned}
L(\mathbf{w}) &= \frac{1}{2}\|\mathbf{y} - \mathbf{X}\mathbf{w}\|^2 \\
&= \frac{1}{2}(\mathbf{y} - \mathbf{X}\mathbf{w})^\top(\mathbf{y} - \mathbf{X}\mathbf{w}) \\
&= \frac{1}{2}\left(\mathbf{y}^\top\mathbf{y} - \mathbf{y}^\top\mathbf{X}\mathbf{w} - \mathbf{w}^\top\mathbf{X}^\top\mathbf{y} + \mathbf{w}^\top\mathbf{X}^\top\mathbf{X}\mathbf{w}\right) \\
&= \frac{1}{2}\left(\mathbf{y}^\top\mathbf{y} - 2\mathbf{w}^\top\mathbf{X}^\top\mathbf{y} + \mathbf{w}^\top\mathbf{X}^\top\mathbf{X}\mathbf{w}\right)
\end{aligned}
$$

Note: We used $\mathbf{y}^\top\mathbf{X}\mathbf{w} = (\mathbf{w}^\top\mathbf{X}^\top\mathbf{y})^\top = \mathbf{w}^\top\mathbf{X}^\top\mathbf{y}$ (scalar)

Computing the Gradient

Recall useful matrix calculus identities:

- $\nabla_{\mathbf{w}}(\mathbf{a}^\top \mathbf{w}) = \mathbf{a}$
- $\nabla_{\mathbf{w}}(\mathbf{w}^\top \mathbf{A}\mathbf{w}) = (\mathbf{A} + \mathbf{A}^\top)\mathbf{w} = 2\mathbf{A}\mathbf{w}$ (if $\mathbf{A}$ symmetric)

Take gradient of $L(\mathbf{w})$:

$$\nabla_{\mathbf{w}} L(\mathbf{w}) = \nabla_{\mathbf{w}} \left[\frac{1}{2}\mathbf{y}^\top \mathbf{y} - \mathbf{w}^\top \mathbf{X}^\top \mathbf{y} + \frac{1}{2}\mathbf{w}^\top \mathbf{X}^\top \mathbf{X}\mathbf{w} \right]$$

$$= 0 - \mathbf{X}^\top \mathbf{y} + \frac{1}{2} \cdot 2(\mathbf{X}^\top \mathbf{X})\mathbf{w}$$

$$= \boxed{\mathbf{X}^\top \mathbf{X}\mathbf{w} - \mathbf{X}^\top \mathbf{y}}$$

Alternative derivation:

$$\nabla_{\mathbf{w}} \|\mathbf{y} - \mathbf{X}\mathbf{w}\|^2 = 2\mathbf{X}^\top (\mathbf{X}\mathbf{w} - \mathbf{y})$$

Solving the Normal Equations

Set gradient to zero for minimum:

$$\nabla_{\mathbf{w}} L(\mathbf{w}) = \mathbf{X}^\top \mathbf{X}\mathbf{w} - \mathbf{X}^\top \mathbf{y} = \mathbf{o}$$

Rearrange:

$$\mathbf{X}^\top \mathbf{X}\mathbf{w} = \mathbf{X}^\top \mathbf{y}$$

These are the **Normal Equations**.

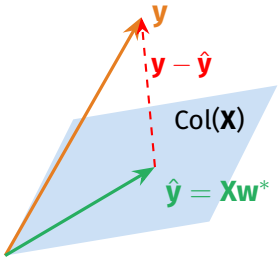
Analytical Solution (OLS Estimator)

$$\boxed{\mathbf{w}^* = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}}$$

Conditions for unique solution:

- $\mathbf{X}^\top \mathbf{X}$ must be **invertible**
- Equivalently: $\mathbf{X}$ has **full column rank**
- Means: No feature is a linear combination of others

Geometric Interpretation



Key Insight:

- $\hat{\mathbf{y}} = \mathbf{X}\mathbf{w}^*$ is the **projection** of $\mathbf{y}$ onto column space of $\mathbf{X}$
- Residual $\mathbf{y} - \hat{\mathbf{y}}$ is **orthogonal** to column space
- This is why it's called **least squares**!

Orthogonality condition:

$$\mathbf{X}^\top(\mathbf{y} - \mathbf{X}\mathbf{w}^*) = \mathbf{o}$$

This is exactly the normal equation!

Proof: The Critical Point is a Minimum

Second-order condition: Check the Hessian matrix

Hessian of $L(\mathbf{w})$:

$$\mathbf{H} = \nabla_{\mathbf{w}}^2 L(\mathbf{w}) = \nabla_{\mathbf{w}}(\mathbf{X}^\top \mathbf{X}\mathbf{w} - \mathbf{X}^\top \mathbf{y}) = \mathbf{X}^\top \mathbf{X}$$

The Hessian is positive semi-definite:
For any vector $\mathbf{v} \neq \mathbf{0}$:

$$\mathbf{v}^\top (\mathbf{X}^\top \mathbf{X})\mathbf{v} = (\mathbf{X}\mathbf{v})^\top (\mathbf{X}\mathbf{v}) = \|\mathbf{X}\mathbf{v}\|^2 \geq 0$$

Conclusion

- $\mathbf{X}^\top \mathbf{X}$ is always positive semi-definite $\Rightarrow$ $L(\mathbf{w})$ is **convex**
- If $\mathbf{X}$ has full rank: $\mathbf{X}^\top \mathbf{X}$ is **positive definite**
- Convex function with unique critical point $\Rightarrow$ **global minimum**

Computational Considerations

Direct Solution Complexity:

- Computing $\mathbf{X}^\top\mathbf{X}$: $O(nd^2)$
- Matrix inversion: $O(d^3)$
- Total: $O(nd^2 + d^3)$

When this is problematic:

- High-dimensional data (d large)
- Very large datasets (n large)
- Need for online/streaming updates

Better numerical methods:

- QR decomposition: more stable
- Cholesky decomposition: if $\mathbf{X}^\top\mathbf{X}$ is well-conditioned
- SVD: handles rank-deficient cases

Key Point

Analytical solutions are nice but **rare** in deep learning. Most models require **iterative optimization**.

Part IV

Gradient Descent

The Workhorse of Deep Learning

The Gradient Descent Idea

Intuition: Walk downhill on the loss surface.

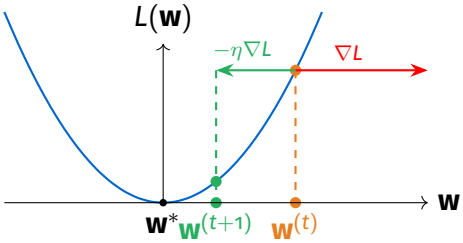
Update Rule:

$$\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla_{\mathbf{w}} L(\mathbf{w})$$

where $\eta > 0$ is the **learning rate**.

Why negative gradient?

- Gradient points uphill (steepest ascent)
- Negative gradient points downhill
- We want to *minimize* loss



Types of Gradient Descent

Batch GD

Use **all** n examples:

$$\nabla L = \frac{1}{n} \sum \nabla \ell^{(i)}$$

Pros: Stable, exact
Cons: Slow for large n

Stochastic GD

Use **one** example:

$$\nabla L \approx \nabla \ell^{(i)}$$

Pros: Fast updates
Cons: Noisy, inefficient

Minibatch SGD

Use $|\mathcal{B}|$ examples:

$$\nabla L \approx \frac{1}{|\mathcal{B}|} \sum \nabla \ell^{(i)}$$

Best of both worlds!
$|\mathcal{B}| \in [32, 256]$

Minibatch SGD balances efficiency with stable updates.

Minibatch SGD for Linear Regression

The Algorithm:

1. Initialize $\mathbf{w}$ randomly, set $b = 0$
2. **For each epoch:**
   1. Shuffle the training data
   2. **For each minibatch** $\mathcal{B}$:
      - Compute gradient on minibatch
      - Update parameters

Update Equations:

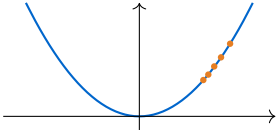
$$\mathbf{w} \leftarrow \mathbf{w} - \frac{\eta}{|\mathcal{B}|} \sum_{i \in \mathcal{B}} \mathbf{x}^{(i)} \left(\mathbf{w}^\top \mathbf{x}^{(i)} + b - y^{(i)} \right)$$

$$b \leftarrow b - \frac{\eta}{|\mathcal{B}|} \sum_{i \in \mathcal{B}} \left(\mathbf{w}^\top \mathbf{x}^{(i)} + b - y^{(i)} \right)$$

Hyperparameters: Learning rate η, batch size $|\mathcal{B}|$, number of epochs

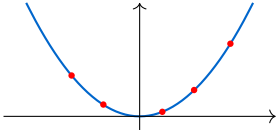
The Learning Rate: A Critical Hyperparameter

Learning Rate Too Small



Very slow convergence!

Learning Rate Too Large



Oscillates or diverges!

Practical Advice

- Start with $\eta = 0.01$ or $\eta = 0.001$
- Use learning rate schedules (decay over time)
- Monitor the loss curve—should decrease smoothly

Part V

Generalization

The Ultimate Goal of Machine Learning

Training Error vs. Generalization Error

Definitions

Training Error R_{emp}: Error on the training dataset

Generalization Error R: Expected error on *unseen* data from the same distribution

Generalization Gap $R - R_{\text{emp}}$: The difference between them

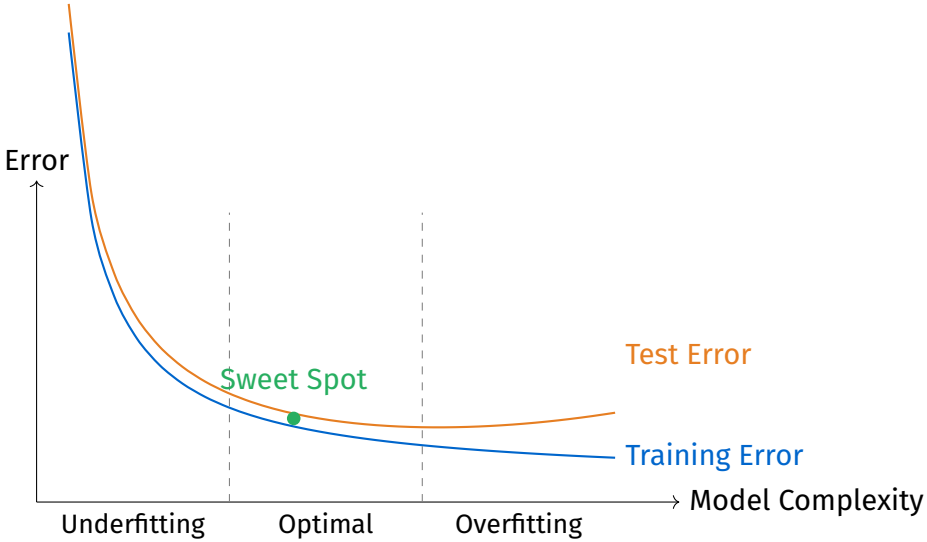
The Central Challenge:

- We can only *measure* training error
- We *care about* generalization error
- Low training error $\not\Rightarrow$ low generalization error!

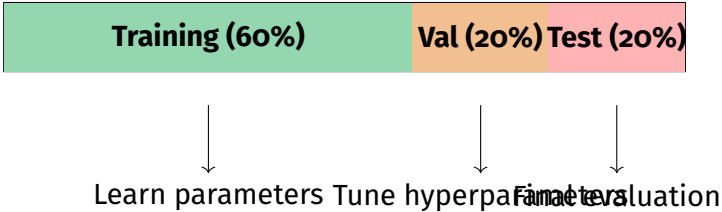
Key Insight

The goal is NOT to minimize training error. The goal is to minimize **generalization error**.

Underfitting and Overfitting



Model Selection: Train/Validation/Test Split



Important Rules:

- **Never** use test data for model selection!
- Use validation set to compare models and tune hyperparameters
- Test set is touched **only once** for final evaluation
- If data is scarce: use **K-fold cross-validation**

Regularization: Weight Decay (L2)

Idea: Penalize large weights to encourage simpler models.

Regularized Loss:

$$L_{\text{reg}}(\mathbf{w}, b) = L(\mathbf{w}, b) + \frac{\lambda}{2}\|\mathbf{w}\|^2$$

where $\lambda \geq 0$ is the **regularization strength**.

Updated Gradient:

$$\nabla_{\mathbf{w}} L_{\text{reg}} = \nabla_{\mathbf{w}} L + \lambda \mathbf{w}$$

SGD Update with Weight Decay:

$$\mathbf{w} \leftarrow \mathbf{w} - \eta(\nabla_{\mathbf{w}} L + \lambda \mathbf{w}) = (1 - \eta\lambda)\mathbf{w} - \eta \nabla_{\mathbf{w}} L$$

Effect

Weights are **shrunk** toward zero at each step — hence "weight decay." This prevents any single feature from having too much influence.

Chapter Summary

1. **Linear Regression** predicts continuous values via $\hat{y} = \mathbf{w}^\top \mathbf{x} + b$
2. **Squared Loss** measures prediction error; emerges from Gaussian noise assumption
3. **Analytical Solution:** $\mathbf{w}^* = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$
   - Requires matrix inversion—expensive and not always possible
4. **Minibatch SGD:** Iteratively update parameters using gradient estimates
   - Scales to large datasets and complex models
   - Foundation for all deep learning optimization
5. **Linear Regression = Single-Layer Neural Network**
   - Inputs directly connected to output
   - Foundation for understanding deeper networks
6. **Generalization** is the ultimate goal—use validation sets and regularization

Key Equations Reference

Concept	Equation
Linear Model	$\hat{y} = \mathbf{w}^\top \mathbf{x} + b$
Matrix Form	$\hat{\mathbf{y}} = \mathbf{X}\mathbf{w} + b$
Squared Loss	$L = \frac{1}{2n}\|\mathbf{y} - \mathbf{X}\mathbf{w}\|^2$
Gradient	$\nabla_{\mathbf{w}} L = \mathbf{X}^\top(\mathbf{X}\mathbf{w} - \mathbf{y})/n$
Normal Equations	$\mathbf{X}^\top\mathbf{X}\mathbf{w} = \mathbf{X}^\top\mathbf{y}$
OLS Solution	$\mathbf{w}^* = (\mathbf{X}^\top\mathbf{X})^{-1}\mathbf{X}^\top\mathbf{y}$
SGD Update	$\mathbf{w} \leftarrow \mathbf{w} - \eta\nabla_{\mathbf{w}} L$

Exercises

1. Given data $x_1, \ldots, x_n \in \mathbb{R}$, find b that minimizes $\sum_i (x_i - b)^2$.
   - What is the analytical solution?
   - How does this relate to the normal distribution?
2. Derive the analytical solution for linear regression *without* absorbing the bias.
3. What happens if $\mathbf{X}^\top \mathbf{X}$ is not invertible?
   - When does this occur?
   - How can regularization help?
4. Implement linear regression from scratch and verify convergence.
5. Experiment with different learning rates. What do you observe?
6. Why can't we simply compose two linear layers to create a "deeper" model?

Thank You!

Questions?

Resources:

`https://d2l.ai`

Interactive notebooks in PyTorch, TensorFlow, JAX

Next Chapter:

Linear Neural Networks for *Classification*

(Softmax Regression)

"The best way to understand deep learning is learning by doing."